



42 LAUSANNE STUDY HANDBOOK

Webserv from Zero

Every dependency question with a concise model answer

Use this book to understand the project before implementing it. The answers explain the required mental model; they are not a substitute for writing, testing, and defending your own code.

How to use this handbook

Study in dependency order. First answer each question without looking. Then compare your explanation with the model answer. A layer is complete only when you can also pass its practical gate and explain one failure case.

Some behaviours are fixed by HTTP or by the 42 subject; others are implementation choices. Where the subject leaves a choice, the answer states the safe design principle and asks you to document the exact behaviour chosen by your team.

	L0 Server mental model	
	L1 Unix bytes and file descriptors	
	L2 Filesystem and processes	
	L3 C++98 engineering	
	L4 TCP and sockets	
	L5 HTTP/1.x	
L6 Configuration	L7 Non-blocking reactor	L8 Incremental parser
	L9 Integrated request lifecycle	
L10A Static site and methods	L10B Bodies and uploads	L10C CGI
	L11 Resilience and cleanup	
	L12 Integration and stress testing	
	L13 Submission and defense	
	Bonus Cookies, sessions, multiple CGI types	

Contents

Layer 0 - Server mental model	4
Layer 1 - Bytes, buffers, system calls, and file descriptors	6
Layer 2 - Filesystem, directories, and processes	9
Layer 3 - C++98 engineering and resource ownership	12
Layer 4 - TCP and sockets	15
Layer 5 - HTTP/1.x syntax, semantics, and framing	18
Layer 6 - Configuration parser and routing model	22
Layer 7 - One non-blocking event loop	25
Layer 8 - Incremental HTTP request parser	28
Layer 9 - Integrated request lifecycle	31
Layer 10A - Static website, routes, and methods	33
Layer 10B - Request bodies and file uploads	36
Layer 10C - CGI	38
Layer 11 - Resilience, backpressure, and cleanup	41
Layer 12 - Integration and stress testing	43
Layer 13 - Submission and defense	45
Bonus - Only after mandatory behaviour is flawless	47
Reference spine	48

Layer 0 - Server mental model

Understand what Webserv is before studying any system call or protocol detail.

L0-Q1. What problem is Webserv trying to solve?

Answer. Webserv implements an HTTP server in C++98. It accepts client connections, receives HTTP requests, selects behaviour from its configuration, serves files or CGI output, accepts uploads, and returns valid HTTP responses while remaining available to other clients.

L0-Q2. What is the difference between a client and a server?

Answer. A client initiates communication because it wants a service or resource. A server waits at a known address and port, accepts those connections, processes requests, and sends responses. In this project a browser, curl, or a test program is the client; Webserv is the server.

L0-Q3. What is a user agent?

Answer. A user agent is software acting for a user when communicating over HTTP. Browsers are common user agents, but crawlers, curl, mobile applications, and automated test clients are user agents too.

L0-Q4. What is an HTTP request, and who sends it?

Answer. An HTTP request is a structured message describing an operation the client wants the server to perform. It contains a request line, headers, and sometimes a body. The client sends it after establishing a connection to the server.

L0-Q5. What is an HTTP response, and who sends it?

Answer. An HTTP response is the server's structured answer. It contains a status line, headers, and possibly a body. Webserv sends it after parsing the request and deciding whether the requested operation succeeded or failed.

L0-Q6. What is a resource? Why is a resource not necessarily a file?

Answer. A resource is the thing identified by a request target. It can correspond to a disk file, a generated directory listing, the result of a CGI program, an upload endpoint, a redirect, or even an error response. A URL identifies server behaviour, not necessarily a literal file.

L0-Q7. What is the difference between static and dynamically generated content?

Answer. Static content already exists, usually as a file, and can be returned without executing application logic. Dynamic content is produced for the request, for example by running a Python CGI script that uses the query string or request body.

L0-Q8. What happens, at a high level, after a user enters `http://localhost:8080/index.html`?

Answer. The browser resolves localhost, opens a TCP connection to port 8080, and sends an HTTP request. Webserv accepts the connection, incrementally reads and parses the request, matches a configured route, obtains or generates the resource, builds a response, sends it, and either keeps the connection available or closes it according to HTTP rules.

L0-Q9. What responsibilities belong to the operating system, and what responsibilities belong to Webserv?

Answer. The kernel manages processes, memory protection, descriptors, TCP delivery, filesystem access, and readiness notifications. Webserv decides application behaviour: HTTP parsing, route selection, file and CGI handling, status codes, buffering, timeouts, and cleanup.

L0-Q10. Why must a web server continue running after responding to one client?

Answer. A server is a long-lived service. Finishing one response completes one request, not the program's purpose. Webserv must return to its event loop so it can serve later requests and simultaneous clients.

Practical pass gate

Draw and explain this complete path without notes: browser -> TCP connection -> Webserv -> HTTP parser -> route -> file, upload, or CGI -> buffered HTTP response -> cleanup or next request.

Layer 1 - Bytes, buffers, system calls, and file descriptors

Learn what the kernel returns to your program and why network I/O can never be treated as complete messages.

L1-Q1. What is the difference between user space and kernel space?

Answer. Webserv runs in user space with restricted access to hardware and protected kernel data. The kernel controls sockets, files, processes, and scheduling. Webserv requests those services through system-call interfaces.

L1-Q2. What is a system call?

Answer. A system call is a controlled request from a user-space program to the operating-system kernel. Functions such as open, read, socket, bind, poll, fork, and execve cross that boundary so the kernel can perform privileged work.

L1-Q3. What is a file descriptor? Why can files, sockets, and pipes all be represented by integers?

Answer. A file descriptor is a process-local integer referring to an open kernel-managed resource. Unix exposes a common I/O interface for many resource types, so read, write, close, and readiness mechanisms can operate on files, sockets, and pipes through descriptors.

L1-Q4. What normally occupies file descriptors 0, 1, and 2?

Answer. Descriptor 0 is standard input, 1 is standard output, and 2 is standard error. New descriptors normally use the lowest available integer, although code must never depend on a particular number.

L1-Q5. What does open() ask the kernel to do, and what does it return?

Answer. open asks the kernel to open or create a filesystem object using specified flags and permissions. On success it returns a new descriptor; on failure it returns -1 and sets errno. The returned descriptor must eventually be closed by its owner.

L1-Q6. What do the arguments and return value of read() mean?

Answer. read(fd, buffer, count) asks for at most count bytes from fd and stores them beginning at buffer. A positive return value is the exact number of bytes actually stored, 0 means end-of-file or an orderly stream close, and -1 means the operation failed.

L1-Q7. Why can read() return fewer bytes than requested?

Answer. count is a maximum, not a promise. A file may have fewer bytes before EOF, a pipe may currently contain only part of the producer's output, or a TCP socket may currently have only some delivered bytes available. A short successful read does not mean the complete application message arrived.

L1-Q8. Why is a short read() not the same as EOF?

Answer. Any positive return means bytes were read and more may arrive later. EOF is represented by a return value of exactly 0. Treating every short read as EOF would truncate fragmented HTTP requests.

L1-Q9. What do return values greater than 0, 0, and -1 from read() mean?

Answer. Greater than 0 is the exact byte count received, 0 means the resource has no more bytes because a file ended or the peer closed its sending side, and -1 indicates failure. The server must interpret each case according to the descriptor and its current state.

L1-Q10. Why are partial reads especially important with TCP?

Answer. TCP is a byte stream and does not preserve HTTP message boundaries. One request may arrive through many reads, and one read may contain parts of multiple requests. Webserv must accumulate bytes and let the parser decide when a request is complete.

L1-Q11. Why can write() or send() write fewer bytes than requested?

Answer. The destination may currently accept only part of the data because its kernel buffer has limited free space or the peer is reading slowly. A positive short write is progress, not failure, and the unsent suffix must remain queued.

L1-Q12. Why must the server remember how many response bytes have already been sent?

Answer. Retrying from the beginning would duplicate bytes, while discarding the buffer would truncate the response. The connection therefore needs an output buffer and an offset indicating the first byte not yet sent.

L1-Q13. Why is a buffer of bytes not necessarily a null-terminated C string?

Answer. read and recv return raw bytes and do not append a terminating zero. The data may also legitimately contain zero bytes, especially in uploads. Code must use the returned length instead of strlen or assumptions about text termination.

L1-Q14. How do you safely accumulate data received through several reads?

Answer. Append exactly the number of bytes returned to a per-connection binary-safe buffer. Preserve unconsumed bytes, track parser progress, and remove data only after the parser has definitively consumed it.

L1-Q15. What is the difference between blocking and non-blocking I/O?

Answer. A blocking operation may suspend the thread until progress is possible. A non-blocking descriptor makes the operation return instead of waiting. Webserv combines non-blocking descriptors with readiness notifications so one slow client cannot stop all others.

L1-Q16. What is the difference between close() and shutdown()?

Answer. close releases this process's descriptor reference; when the last reference disappears, the underlying socket is destroyed. shutdown changes one or both communication directions of a connected socket while the descriptor can remain open, which is useful for half-closing a stream.

L1-Q17. Which descriptors can wait indefinitely for data?

Answer. Network sockets, pipes, FIFOs, and similar streams may have no data yet while remaining open. A blocking read on them can wait indefinitely, so the subject requires non-blocking, readiness-driven I/O for these resources.

L1-Q18. Why does the subject exempt regular disk files from readiness monitoring?

Answer. Readiness APIs generally report regular files as immediately ready because their data is addressed through the filesystem rather than arriving asynchronously like network or pipe data. The subject therefore permits direct reads and writes on regular files.

Practical pass gate

Explain this sequence precisely: recv returns 12, then 5, then 28, then 0. State what is appended after every call, why no positive result proves request completion, and why 0 signals peer closure rather than an empty HTTP message.

Layer 2 - Filesystem, directories, and processes

Connect URL-to-file behaviour with Unix paths, then learn the process and pipe primitives later required by CGI.

L2-Q1. What is the difference between an absolute path and a relative path?

Answer. An absolute path begins at the filesystem root, such as `/var/www/index.html`. A relative path is interpreted from the process's current working directory. Webserv should construct predictable paths and never let untrusted URL components control that base.

L2-Q2. What is the current working directory?

Answer. It is the directory used as the base for relative path resolution by the process. Every process has one, inherited across fork and changed by `chdir`.

L2-Q3. What does `chdir()` change?

Answer. `chdir` changes the calling process's current working directory. In CGI, the child should change to the required script directory so relative file access behaves correctly, while the parent server should preserve its own stable working context.

L2-Q4. What information can `stat()` provide?

Answer. `stat` reports metadata such as object type, size, permissions, ownership, and timestamps. Webserv can use it to distinguish regular files from directories and to determine whether a requested resource exists in an expected form.

L2-Q5. How do `access()`, `stat()`, and `open()` differ?

Answer. `access` checks permissions using process credentials, `stat` obtains metadata, and `open` actually requests a usable descriptor. A prior access check cannot guarantee a later open will succeed, so the result of the real operation remains authoritative.

L2-Q6. How do `opendir()`, `readdir()`, and `closedir()` work together?

Answer. `opendir` opens a directory stream, `readdir` returns entries one at a time until completion, and `closedir` releases it. Webserv can use these functions to generate an autoindex while filtering and escaping entries safely.

L2-Q7. How can the server determine whether a path is a file or directory?

Answer. Call `stat` and inspect the file-type bits with the associated macros. Do not guess based on a trailing slash or filename extension because names do not determine filesystem type.

L2-Q8. What are . and .., and why can .. become a security problem?

Answer. . names the current directory and .. names its parent. If client-controlled paths containing .. are joined naively to a configured root, the request may escape that root and expose or delete unrelated files.

L2-Q9. Why must URL paths be normalized before being mapped to disk paths?

Answer. Normalization resolves path segments consistently and permits a final containment check against the configured root. It also prevents equivalent encoded or repeated-separator paths from bypassing routing and security checks.

L2-Q10. What should happen if a requested directory has an index file?

Answer. If the matched route defines an index and that file exists and is allowed, Webserv should serve it as the directory's representation. The configured index lookup happens before autoindex generation.

L2-Q11. What should happen if a directory has no index file but directory listing is enabled?

Answer. Webserv should generate an HTML listing of allowed entries with correctly escaped display text and URL-encoded links. If listing is disabled, it should return the configured error behaviour rather than expose directory contents.

L2-Q12. What is a process?

Answer. A process is a running program with its own virtual address space, descriptor table, credentials, signal state, and execution context. Webserv is one process; each CGI invocation is executed in a child process.

L2-Q13. What does fork() create, and what does it return in each process?

Answer. fork creates a child process initially copying the parent's execution state and descriptor references. It returns the child's PID in the parent, 0 in the child, and -1 if no child was created.

L2-Q14. What does execve() do to the current process?

Answer. execve replaces the calling process image with another executable while keeping the same PID. On success it never returns; on failure the original process continues after execve and must report or exit appropriately.

L2-Q15. What is a pipe, and why does it have two ends?

Answer. A pipe is a one-directional byte stream managed by the kernel. One descriptor is read from and the other is written to. Full duplex CGI communication therefore normally needs two pipes: one toward the child and one back to the parent.

L2-Q16. What do dup() and dup2() do?

Answer. They create another descriptor referring to the same open resource. dup2 can choose the target descriptor number, which lets a CGI child connect a pipe to standard input or standard output before execve.

L2-Q17. Why does CGI require one channel for input and another for output?

Answer. The server must send the request body into the CGI program while independently reading the CGI response. Because a normal pipe is one-way, these directions need separate pipes and must be handled concurrently to avoid deadlock.

L2-Q18. What is a zombie process, and how does `waitpid()` prevent it?

Answer. After a child exits, the kernel retains its termination status until the parent collects it. That unreaped entry is a zombie. `waitpid` collects the status; Webserv should use a non-blocking mode so collecting children cannot freeze the event loop.

L2-Q19. Why does the subject forbid `fork()` outside CGI?

Answer. The required concurrency model is one non-blocking event loop, not one process per client. Restricting `fork` to CGI forces the team to understand readiness, per-connection state, and multiplexed I/O.

Practical pass gate

Explain how a parent creates two pipes, forks, connects the child's standard input and output with `dup2`, closes every unused end, executes a CGI program, feeds its body, reads its output, detects exit, and reaps the child without blocking.

Layer 3 - C++98 engineering and resource ownership

Design explicit ownership and state so errors cannot leak descriptors or corrupt unrelated connections.

L3-Q1. What restrictions does compiling with `-std=c++98` impose?

Answer. The project cannot depend on C++11 or later features such as move semantics, range-for, auto type deduction, unordered containers, smart pointers from the standard library, lambdas, or nullptr. All design and dependencies must compile under the C++98 language and library.

L3-Q2. Which C++ containers are useful for listeners, connections, headers, routes, and buffers?

Answer. vector is useful for ordered collections and poll descriptors, map for lookup by descriptor or normalized header name, string for binary-safe byte storage when lengths are respected, and deque or list when stable insertion and removal properties are specifically useful. Choose by required semantics, not habit.

L3-Q3. What is RAI?

Answer. Resource Acquisition Is Initialization ties a resource's lifetime to an object's lifetime. A constructor acquires or receives ownership, and the destructor releases it. In Webserv this can make descriptor and directory cleanup reliable across normal returns and exceptions.

L3-Q4. Who owns each listening socket, client socket, pipe, and file?

Answer. The architecture must assign exactly one closing responsibility for every descriptor. For example, the server owns listeners, each Connection owns its client socket and file, and a CGI context owns its child pipes and PID. Other components may reference them but must not close them independently.

L3-Q5. When must each descriptor be closed?

Answer. Close it when its job is complete, when the associated connection or child fails, or during server shutdown. Every state transition that abandons work needs a cleanup path, and ownership should make repeated cleanup harmless rather than causing double close.

L3-Q6. Why can copying an object that owns a file descriptor cause a double-close bug?

Answer. A normal integer copy makes both objects contain the same descriptor number without creating independent ownership. If both destructors close it, the second close may target an unrelated resource that has reused that number.

L3-Q7. Should an owning object be copyable? If so, what should copying mean?

Answer. The safest C++98 default is to make exclusive owners non-copyable. If copying is genuinely required, it must have explicit semantics, such as duplicating the descriptor with `dup` or transferring ownership through a documented manual operation; implicit integer copying is not ownership management.

L3-Q8. What information belongs in a Connection object?

Answer. Typical state includes the client descriptor, listener or server identity, remote information if needed, input and output buffers, send offset, parser and request state, selected route, deadlines, keep-alive decision, current file or CGI work, and a closing or failure reason.

L3-Q9. Why are composition and explicit state usually more useful than deep inheritance here?

Answer. The main complexity comes from data flow and lifecycle states rather than many substitutable object families. Small composed components such as parser, router, response builder, and CGI context make ownership and transitions easier to inspect and test.

L3-Q10. How should errors propagate without crashing the complete server?

Answer. Local failures should become a connection error, HTTP error response, CGI failure, or startup configuration failure as appropriate. Catch exceptions at controlled boundaries, release owned resources, and keep unrelated connections in the event loop.

L3-Q11. How could allocation failure be contained instead of terminating unexpectedly?

Answer. Catch `std::bad_alloc` at high-level request or event-loop boundaries, avoid further large allocations, close or reject the affected connection, and release resources predictably. No program can manufacture memory when the system is fully exhausted, but it can avoid uncaught exceptions and uncontrolled corruption.

L3-Q12. What must the Makefile rules `all`, `clean`, `fclean`, `re`, and `NAME` do?

Answer. `NAME` identifies the webserv executable; `all` builds it; `clean` removes intermediate objects; `fclean` removes objects and the executable; and `re` performs a complete rebuild. Dependencies and timestamps must prevent needless recompilation or relinking.

L3-Q13. What is unnecessary relinking?

Answer. It is relaunching the final link command when no relevant source, object, or build input changed. Correct Makefile dependencies let make determine that an already up-to-date executable needs no work.

L3-Q14. Why should parsing, networking, routing, and response generation be separate components?

Answer. They have different invariants and can fail independently. Separation permits unit tests, avoids mixing protocol syntax with system-call control flow, and lets the event loop coordinate components without each component secretly owning the entire connection.

Practical pass gate

Design an FD owner, Connection, Request, Response, Route, and CGI context. For every descriptor and buffer, state who owns it, whether copying is allowed, which state uses it, and exactly where cleanup happens on success and failure.

Layer 4 - TCP and sockets

Understand the transport Webserv uses before adding HTTP semantics on top of it.

L4-Q1. What are an IP address, an interface, and a port?

Answer. An IP address identifies a network endpoint at the Internet layer, an interface is a host's connection to a network and owns one or more addresses, and a port identifies a particular transport service on that host. Webserv binds listening sockets to configured address-and-port pairs.

L4-Q2. What identifies a TCP connection?

Answer. A TCP connection is distinguished by protocol plus the local IP and port and remote IP and port. This allows many clients to connect to the same listening port at the same time while each accepted connection remains distinct.

L4-Q3. What is network byte order, and why do htons() and ntohs() exist?

Answer. Protocols encode multi-byte integers in a standard big-endian order called network byte order. htons converts a 16-bit host value such as a port into network order, while ntohs converts it back; the analogous long functions handle 32-bit values.

L4-Q4. What does getaddrinfo() provide?

Answer. It translates host, service, and socket requirements into one or more address structures suitable for socket operations. It handles address-family details so code can iterate valid candidates instead of manually constructing every sockaddr variant.

L4-Q5. What does socket() create?

Answer. socket creates a kernel communication endpoint and returns its descriptor. It is not connected or listening yet; later calls define its address and role.

L4-Q6. What does setsockopt() configure? Why is address reuse useful?

Answer. setsockopt changes selected socket behaviour. SO_REUSEADDR is commonly set before bind so a recently used server address can be rebound after restart when old TCP state would otherwise delay it; it does not permit arbitrary duplicate active listeners.

L4-Q7. What does bind() associate with a socket?

Answer. bind associates the socket with a local interface address and port. A failure may mean the address is unavailable, already in use, forbidden, or malformed, and startup should report that clearly.

L4-Q8. What does listen() change about a socket?

Answer. listen marks a bound stream socket as passive, meaning it can receive incoming connection attempts. The socket then represents a listening endpoint rather than one client conversation.

L4-Q9. What is the listen backlog?

Answer. It is the requested limit associated with pending connections waiting to be accepted. The kernel applies platform-specific limits, so it is not the maximum number of clients the server can ever handle.

L4-Q10. What does accept() return?

Answer. accept removes one completed connection from a listening socket's queue and returns a new descriptor representing that client connection. The original listener remains open for future connections.

L4-Q11. Why are the listening descriptor and accepted client descriptor different?

Answer. The listener represents a local service endpoint shared by all incoming clients, while the accepted descriptor represents one specific TCP byte stream. Closing a client must not close the listener, and accepting another client must not overwrite existing connection state.

L4-Q12. Why does every connected client need its own state?

Answer. Clients progress independently: one may be reading headers, another running CGI, and another sending a large response. Each requires separate buffers, parser state, deadlines, routing decisions, and output offsets.

L4-Q13. Why does TCP not preserve application-message boundaries?

Answer. TCP promises an ordered stream of bytes, not packets that match send calls or HTTP messages. The network stack can split, combine, delay, and buffer bytes while preserving their order.

L4-Q14. Can one recv() be assumed to contain one complete HTTP request?

Answer. No. It can contain one byte, half a header, a complete request, or more than one request. Completion must be established by HTTP framing rules, never by the number of recv calls.

L4-Q15. Can one recv() contain multiple HTTP requests?

Answer. Yes, especially on a persistent connection when requests are sent close together. After consuming one complete request, the parser must preserve any remaining bytes for the next request.

L4-Q16. How does the server detect an orderly client disconnection?

Answer. Once read readiness is reported, recv returning 0 indicates that the peer has closed its sending side. Webserv then decides whether pending output can or should finish and releases the connection according to its state.

L4-Q17. How can Webserv listen on multiple ports simultaneously?

Answer. Create, configure, bind, and listen on one socket for every unique configured interface-and-port pair. Register all listening descriptors in the same event multiplexer and associate each with its server configuration.

L4-Q18. What is the difference between TCP connection establishment and HTTP processing?

Answer. TCP establishes a reliable byte stream between endpoints. HTTP defines how bytes inside that stream form requests and responses. accept completes the server side of transport setup; only then can Webserv parse HTTP data.

Practical pass gate

Explain or build a learning-only echo server using socket -> setsockopt -> bind -> listen -> accept -> recv/send -> close. Then identify every place where that blocking design would freeze other clients and why it cannot be the final architecture.

Layer 5 - HTTP/1.x syntax, semantics, and framing

Learn exactly how raw bytes become requests and how responses communicate success, failure, metadata, and body boundaries.

L5-Q1. What are the components of an HTTP request?

Answer. A request contains a request line, zero or more header fields, an empty line, and an optional body whose existence and length are determined by framing headers. The empty line terminates the header section.

L5-Q2. What are the three components of the request line?

Answer. They are the method, request target, and HTTP version, for example GET /index.html HTTP/1.1. Their syntax must be validated before routing.

L5-Q3. What separates the method, target, and HTTP version?

Answer. The normal request-line grammar uses single space characters between the three components and ends with CRLF. A robust strict parser should not silently accept arbitrary whitespace forms unless that behaviour is deliberately supported.

L5-Q4. Why are HTTP lines normally terminated with CRLF?

Answer. HTTP/1.x message syntax defines a carriage return followed by a line feed as its line terminator. The parser searches for that two-byte sequence rather than relying on platform text-file conventions.

L5-Q5. Why is there an empty line after the headers?

Answer. It marks the end of the header section. Without this explicit delimiter, the receiver could not know whether following bytes were another header or the beginning of the body.

L5-Q6. What are the components of an HTTP response?

Answer. A response contains a status line, response headers, an empty line, and an optional body. Certain methods and status codes prohibit a body even if other responses normally include one.

L5-Q7. What are the three components of the status line?

Answer. They are the HTTP version, a three-digit status code, and a reason phrase, for example HTTP/1.1 200 OK. Clients primarily rely on the numeric code; the phrase is explanatory text.

L5-Q8. Are header names case-sensitive?

Answer. No. Header field names are case-insensitive, so Content-Length and content-length identify the same field. Values have header-specific rules and must not all be lowercased.

L5-Q9. What is the difference between the URL path and query string?

Answer. The path selects a hierarchical resource or route. The query begins after ? and provides additional parameters; it is not part of the filesystem path. CGI receives it separately through QUERY_STRING.

L5-Q10. What are the intended meanings of GET, POST, and DELETE?

Answer. GET retrieves a representation and should not change server state merely by being read. POST submits data for route-defined processing such as upload or CGI. DELETE requests removal of the target resource according to the server's documented policy.

L5-Q11. How does a server know whether a request has a body?

Answer. It uses HTTP framing, principally Transfer-Encoding or Content-Length for this project, together with request validity rules. The method name alone does not tell the parser how many body bytes follow.

L5-Q12. What does Content-Length describe?

Answer. It is the decimal number of body octets in that HTTP message. It does not include the request line, headers, empty separator line, or bytes belonging to a later message.

L5-Q13. How does Transfer-Encoding: chunked frame a body?

Answer. The body is a sequence of chunks. Each begins with a hexadecimal size line, followed by exactly that many data bytes and CRLF. A zero-size chunk ends the data and can be followed by permitted trailers and a final empty line.

L5-Q14. What marks the end of a chunked body?

Answer. A parsed chunk size of zero marks the end of chunk data. The message completes only after consuming the zero chunk's line, any trailer fields, and the terminating empty line.

L5-Q15. What should happen if framing headers contradict one another?

Answer. Reject the request as malformed rather than guessing. Conflicting Content-Length values or an unsafe combination of Content-Length and Transfer-Encoding can create request-smuggling ambiguities between components.

L5-Q16. Why must chunked bodies be decoded before being sent to CGI?

Answer. CGI expects the entity body itself on standard input and EOF after it, not HTTP chunk-size lines and delimiters. Webserv removes transfer framing and supplies the resulting body length and content metadata to the child.

L5-Q17. What is a persistent connection or keep-alive?

Answer. It is a TCP connection reused for more than one HTTP request-response exchange. Webserv must preserve leftover bytes, reset request-specific state, and apply the correct version and Connection rules before accepting another request on it.

L5-Q18. How do HTTP/1.0 and HTTP/1.1 differ regarding persistent connections?

Answer. HTTP/1.0 normally closes after a response unless keep-alive is explicitly negotiated. HTTP/1.1 normally keeps the connection persistent unless Connection: close or framing and error conditions require closure.

L5-Q19. Why is Host important in HTTP/1.1 even though virtual hosting is optional here?

Answer. Host identifies the requested authority and is required by normal HTTP/1.1 request syntax. Webserv may map only by interface and port because virtual hosts are out of scope, but it should still validate its chosen HTTP/1.1 requirements consistently.

L5-Q20. When should the server return 200, 201, or 204?

Answer. 200 indicates a successful response with a representation or general result. 201 indicates that processing created a resource, commonly an upload. 204 indicates successful processing with no response body, often suitable for a successful deletion.

L5-Q21. What is the difference between 301, 302, 307, and 308?

Answer. 301 and 308 are permanent; 302 and 307 are temporary. 307 and 308 explicitly preserve the request method and body, while clients historically may transform POST into GET after 301 or 302. Every redirect needs a Location header.

L5-Q22. What is the difference between 400, 403, and 404?

Answer. 400 means the request itself is malformed. 403 means the server understood the target but refuses access. 404 means no matching resource is available, although a server may intentionally use 404 to avoid revealing a forbidden resource.

L5-Q23. What is the difference between 405 Method Not Allowed and 501 Not Implemented?

Answer. 405 means the server recognizes the method but the selected resource does not permit it, and the response should include Allow. 501 means the server does not implement or recognize the requested method capability at all.

L5-Q24. When should 411 Length Required or 413 Content Too Large be returned?

Answer. 411 applies when the route requires a framed body but the request supplies no acceptable way to determine its length. 413 applies when the declared or cumulatively decoded body exceeds the configured maximum.

L5-Q25. When are 500, 502, and 504 appropriate?

Answer. 500 represents an internal server failure. 502 is appropriate when a gateway-like component such as CGI produces an invalid or unusable response under the server's contract. 504 represents a timeout waiting for such upstream processing.

L5-Q26. What do Content-Type, Location, Allow, and Connection communicate?

Answer. Content-Type describes the body's media type; Location supplies a redirect or created-resource URI; Allow lists methods valid for the resource; and Connection carries hop-specific connection controls such as close.

L5-Q27. How does the client determine where a response body ends?

Answer. The status and request method can imply no body; otherwise Content-Length, chunked transfer coding, or closure of the connection delimits it. A persistent connection requires explicit framing because closure cannot mark each response boundary.

L5-Q28. Which malformed requests must be rejected rather than partially accepted?

Answer. Reject invalid request lines or versions, illegal header syntax, invalid numeric lengths, contradictory framing, malformed chunks, forbidden control characters, and requests exceeding defined limits. The parser must never reinterpret ambiguous bytes as a second harmless request.

Practical pass gate

Using a raw client, manually send a GET, a Content-Length POST, and a chunked request. Annotate every delimiter and framing byte, predict each response, and compare important behaviour with NGINX while accounting for HTTP-version differences.

Layer 6 - Configuration parser and routing model

Turn the subject's configurable behaviour into a deterministic, validated in-memory model.

L6-Q1. What configuration syntax will your team define?

Answer. The subject permits an NGINX-inspired syntax but does not prescribe one. The team must define a small unambiguous grammar for server blocks, route blocks, directives, values, separators, and comments, then document it with valid examples.

L6-Q2. How will the configuration be tokenized and parsed?

Answer. A tokenizer should convert characters into meaningful tokens while tracking positions for errors. A parser should then enforce the grammar and build temporary configuration objects. Separating these stages makes syntax errors testable and avoids ad hoc string searching.

L6-Q3. What makes a configuration syntactically valid but semantically invalid?

Answer. Its token structure may follow the grammar while its meaning is impossible or contradictory, such as an invalid port, duplicate incompatible directive, negative body limit, empty method list, inaccessible CGI executable, or overlapping duplicate listener definition.

L6-Q4. How will interface-and-port pairs be represented?

Answer. Use a value object containing the resolved address family, binary or canonical interface address, port, and a link to the server configuration. Equality must be defined so duplicate listener endpoints can be detected before socket creation.

L6-Q5. How will multiple listening ports be represented?

Answer. A server block can hold a collection of listen endpoints, or normalized startup data can create one listener record per endpoint. The event loop then associates every listening descriptor with the configuration used for accepted clients.

L6-Q6. How will default error pages be configured?

Answer. Store a mapping from status code to validated file path at the relevant server scope. Response generation first tries that custom page and falls back to a built-in generated page if it cannot be used safely.

L6-Q7. Where is the maximum request-body size stored?

Answer. Store a parsed numeric byte limit in the effective configuration used by a request. If route-level overrides are supported, resolve inheritance once or provide a deterministic lookup so reception can enforce the limit before the full body arrives.

L6-Q8. How will a route store accepted methods?

Answer. Use a normalized set or bit-like representation for methods supported by the project. Route matching selects the route first; method authorization then either continues or creates a 405 response with the corresponding Allow value.

L6-Q9. How will redirects be represented?

Answer. Store the selected redirect status and destination as route data. Once that route matches, response planning can create the status and Location header without attempting filesystem or CGI processing.

L6-Q10. How will root, directory listing, and index files be represented?

Answer. The route needs a filesystem root, an explicit autoindex boolean, and one or more configured index names according to your grammar. These are separate properties because a directory can have an index lookup even when listing is disabled.

L6-Q11. How will upload authorization and destination be represented?

Answer. Store whether direct upload is allowed and a validated destination directory. Treat the destination as server-controlled configuration; never derive the storage base directly from an untrusted filename or URL segment.

L6-Q12. How will CGI file extensions and executables be associated?

Answer. Use a mapping from normalized extension to interpreter or executable information. Route resolution and filesystem mapping find the script first, then the CGI mapping determines the child command and environment.

L6-Q13. If /kapouet maps to /tmp/www, where should /kapouet/a/b map?

Answer. Remove the matched route prefix and append the remaining normalized path to the configured root, producing /tmp/www/a/b. The final normalized result must remain inside /tmp/www.

L6-Q14. If several route prefixes match, which one wins?

Answer. Use the longest valid path-prefix match, not first declaration by accident. Matching should respect segment boundaries so a route /app does not unintentionally match /apple unless your documented syntax explicitly says it does.

L6-Q15. What should happen with duplicate or contradictory directives?

Answer. Define which directives may repeat and reject all unsupported duplicates with a line-aware startup error. Silently taking the first or last value hides configuration mistakes and makes behaviour dependent on parser accidents.

L6-Q16. What should happen if an error page or upload directory is inaccessible?

Answer. Choose and document whether startup rejects the configuration or runtime falls back safely. Critical destinations such as an upload directory are best validated at startup, while an unavailable custom error page can reasonably fall back to the built-in page.

L6-Q17. Should invalid configuration be rejected before any socket starts listening?

Answer. Yes. Parse and validate the entire configuration first so startup is atomic: either the server has a coherent model and begins listening, or it reports errors and exits without becoming partially available.

L6-Q18. How does the optional command-line argument interact with the default configuration path?

Answer. If exactly one path argument is supplied, parse that file. If none is supplied, use the documented default path. Invalid argument counts or unreadable configuration must produce a clear startup error rather than silently using another file.

L6-Q19. Why do multiple interface-and-port servers not require Host-based virtual hosting?

Answer. The listener that accepted the connection already identifies the configured address-and-port service. Host-based selection lets multiple sites share one endpoint, but the subject explicitly considers that virtual-host feature out of scope.

Practical pass gate

Given one configuration, predict at least ten requests across ports, route prefixes, methods, redirects, directories, uploads, CGI, and missing resources. Then create invalid files that exercise every semantic validation rule and confirm startup is atomic.

Layer 7 - One non-blocking event loop

Build the central reactor required by the subject: one readiness multiplexer for every socket and pipe that can wait.

L7-Q1. Why can one blocking client freeze a single-threaded server?

Answer. If the only thread blocks waiting for that client's input or output, it cannot return to accept new clients or process events for existing ones. The other clients may be ready, but no code runs to serve them.

L7-Q2. What does setting a descriptor to non-blocking mode change?

Answer. An operation that would otherwise wait returns without sleeping the thread. Non-blocking mode does not make data arrive sooner; it lets the central loop remain in control and try operations only after readiness notification.

L7-Q3. What does I/O readiness mean?

Answer. It means an operation of the indicated class can make progress without the normal indefinite wait at that moment. Read readiness may mean bytes or EOF; write readiness means some output-buffer capacity, not necessarily capacity for the complete response.

L7-Q4. Does ready mean the entire operation will finish?

Answer. No. A readable socket may yield only part of a request, and a writable socket may accept only part of a response. Readiness authorizes an attempt; return values describe the actual progress.

L7-Q5. What does poll() monitor?

Answer. poll receives a collection of descriptors and requested event masks, waits up to a timeout, and reports which events occurred. Webserv uses those results to decide which non-blocking operation is permitted next.

L7-Q6. Which descriptors must participate in the single central poll?

Answer. All I/O that can wait: listening sockets, connected client sockets, and CGI pipe ends used for non-blocking reads or writes. Regular disk files are exempt under the subject.

L7-Q7. Why must listening sockets also go through the multiplexer?

Answer. accept can wait when no connection is pending. Polling the listener first keeps the one-loop design non-blocking and satisfies the subject's explicit requirement that listen-side I/O be included.

L7-Q8. When may Webserv call accept()?

Answer. After the corresponding listening descriptor reports read readiness. Each accepted descriptor must immediately receive non-blocking configuration, connection state, and registration in the same event system.

L7-Q9. When may it call `recv()` or `read()` on a socket or pipe?

Answer. Only after that descriptor reports read readiness through the central multiplexer. The call's return value is then used to append bytes, detect EOF, or handle failure without treating `errno` as the readiness mechanism.

L7-Q10. When may it call `send()` or `write()` on a socket or pipe?

Answer. Only when output is pending and the descriptor reports write readiness. After the call, advance by exactly the positive number of bytes written and retain any remainder.

L7-Q11. How can reading and writing be monitored simultaneously?

Answer. A client descriptor can request both readable and writable events when both directions currently matter. The event mask is recalculated from connection state, so write interest appears only while queued output exists.

L7-Q12. Why should `POLLOUT` normally be requested only when output is pending?

Answer. Sockets are often writable, so permanent `POLLOUT` interest can make poll return immediately in a tight loop, wasting CPU. Enabling it only for queued bytes turns readiness into useful work.

L7-Q13. What information maps a ready descriptor back to its connection or CGI process?

Answer. Maintain an explicit registry keyed by descriptor or by stable event entries. Each record identifies its role, owner, requested events, and state, so the dispatcher never guesses whether an `fd` is a listener, client, CGI input, or CGI output.

L7-Q14. How are partial reads and writes represented in connection state?

Answer. Input bytes remain in an accumulation buffer with parser progress. Output bytes remain in a response buffer with a send offset. The event mask reflects whether more input is expected or more output remains.

L7-Q15. What is backpressure?

Answer. Backpressure is the need to slow or bound production when a downstream consumer cannot accept data quickly enough. For example, Webserv must not let unlimited CGI output accumulate because a client is reading slowly.

L7-Q16. What should happen when a client stops reading its response?

Answer. Keep only bounded queued output, wait for future write readiness, and enforce a response deadline. If the client remains stalled or exceeds limits, close it and terminate or detach related work safely.

L7-Q17. How are hangups and descriptor errors handled?

Answer. Interpret poll's error and hangup flags together with any readable final data, mark the owner for controlled cleanup, and never reuse stale event entries. Pending CGI or file resources must be released with the connection.

L7-Q18. How can a descriptor be closed safely while iterating through poll results?

Answer. Defer structural removals, mark entries dead, or use an index-safe removal strategy so iterator invalidation cannot skip or reinterpret events. Remove the registry entry and poll entry consistently before the descriptor number can be reused.

L7-Q19. How do you prevent one busy client from monopolizing the loop?

Answer. Perform bounded work per ready event, return to the dispatcher, and avoid unbounded parse, accept, read, or write loops. Fairness limits and time budgets let other ready descriptors progress.

L7-Q20. Why must regular files and socket or pipe I/O be treated differently under the subject?

Answer. Sockets and pipes may wait for external producers or consumers, so they must be non-blocking and readiness-driven. Regular disk files are explicitly exempt and can be read or written directly, although large synchronous disk work can still affect latency.

L7-Q21. Why does the subject forbid using errno after read() or write() as normal control flow?

Answer. It wants readiness, not a speculative failed read or write, to decide when I/O is attempted. The design must call these operations only after the multiplexer authorizes them and use actual progress and connection state rather than probing until errno reports would-block.

L7-Q22. Which fcntl flags are permitted by the macOS-specific rule?

Answer. The subject limits its use to the listed non-blocking and close-on-exec configuration: F_SETFL, O_NONBLOCK, and FD_CLOEXEC. Follow the exact subject and evaluation interpretation for the target platform.

L7-Q23. How will the event loop guarantee that no request hangs indefinitely?

Answer. Store deadlines for connection phases, give poll a finite timeout based on the next deadline, and scan expired work after wake-up. Timeout handling must build an error when possible or close and clean up when no valid response can be sent.

Practical pass gate

Simulate this event trace: a listener becomes readable; two clients connect; A sends half a request; B sends a complete request; B becomes writable but accepts only part of its response; A disconnects; B becomes writable again. For every step, name the descriptor role, authorized operation, return-value handling, state change, and new event mask.

Layer 8 - Incremental HTTP request parser

Make parsing independent of TCP fragmentation and resistant to ambiguous or excessive input.

L8-Q1. Why must parsing be incremental?

Answer. TCP can deliver any prefix of a request at a time. An incremental parser consumes what is available, records its state, and resumes when more bytes arrive instead of requiring one complete message in memory from one recv call.

L8-Q2. What happens when CRLF is divided between two reads?

Answer. The input buffer preserves the trailing partial delimiter. After the next append, the parser searches or continues its delimiter state and recognizes the complete header terminator without losing or duplicating bytes.

L8-Q3. What happens when one read contains headers plus only part of the body?

Answer. Parse and validate the headers, determine the framing and effective size limit, then change to the appropriate body state. Consume the available body prefix and wait for the remaining declared or chunked data.

L8-Q4. What happens when one read contains the end of one request and the beginning of another?

Answer. Consume exactly the first message's framed length and leave the suffix untouched. After the first response lifecycle permits another request, reset request-specific parser state and continue with those leftover bytes.

L8-Q5. Which parser states are needed for headers, fixed-length bodies, and chunked bodies?

Answer. At minimum use states for request line and headers or a bounded header block, fixed-length body, chunk-size line, chunk data, post-chunk CRLF, trailers, complete, and error. Explicit states prevent one branch from reinterpreting the same byte differently.

L8-Q6. How do you determine that the headers are complete?

Answer. Recognize the first valid CRLF sequence after a syntactically valid request line and header lines. Do not use a lone newline or buffer exhaustion as completion.

L8-Q7. What limits prevent an attacker from sending unlimited headers?

Answer. Set maximum request-line, individual-header, header-count, and total-header sizes, plus a deadline for completion. Reject the connection with an appropriate response or close it once a limit is exceeded.

L8-Q8. How are the method, target, and version validated?

Answer. Validate token characters and supported method policy, target form and forbidden characters, exact version syntax, and defined size limits. Parsing a string into three pieces is not enough; each piece has protocol constraints.

L8-Q9. How are header names normalized without damaging header values?

Answer. Validate the field name and store a lowercase copy as the lookup key because names are case-insensitive. Preserve the value's meaningful case and bytes after trimming only grammar-defined surrounding whitespace.

L8-Q10. How should duplicate Content-Length headers be handled?

Answer. Never choose one silently. A conservative server can reject duplicates; a more RFC-aware policy may accept only values that are all syntactically valid and identical. Any difference is a 400 error because it creates ambiguous framing.

L8-Q11. How should Content-Length combined with chunked encoding be handled?

Answer. Reject the request under a strict safe policy. Accepting conflicting framing interpretations can let an attacker make Webserv and another HTTP component disagree about where one request ends.

L8-Q12. How is a hexadecimal chunk size parsed safely?

Answer. Validate every permitted digit, stop at the grammar-defined extension delimiter if extensions are supported, detect numeric overflow before multiplication or addition, and enforce the cumulative body limit before accepting the chunk data.

L8-Q13. How is the final zero-sized chunk detected?

Answer. After parsing a valid chunk-size line with numeric value zero, switch to trailer processing. Complete the body only after permitted trailers and their terminating empty line have been consumed.

L8-Q14. Why must request bodies remain binary-safe?

Answer. Uploads and CGI input can contain any byte, including zero and CRLF sequences. Their boundaries come from HTTP framing, not string termination or scanning for text delimiters inside body data.

L8-Q15. When exactly is a request considered complete?

Answer. After valid headers and exactly the framed body have been consumed: immediately after headers for a zero-length body, after Content-Length bytes, or after the complete zero chunk and trailer terminator for chunked input.

L8-Q16. What should happen to bytes remaining after a request is complete?

Answer. Keep them in the connection input buffer as possible bytes of the next request. Never discard them and never attach them to the completed body.

L8-Q17. Why should syntax parsing, semantic validation, and routing be separate steps?

Answer. Syntax determines whether bytes form a valid message, semantic validation checks protocol meaning and limits, and routing applies configuration. Separation prevents malformed input from influencing filesystem paths and lets each layer be tested independently.

L8-Q18. How is the configured body-size limit enforced during incremental reception?

Answer. Reject a fixed declared length that already exceeds the limit. For chunked or streamed bodies, maintain a checked cumulative decoded count and reject before appending or writing any chunk that would cross the maximum.

Practical pass gate

Feed one request all at once, one byte at a time, split inside the request line, between CR and LF, inside Content-Length, and at every chunk boundary. Every fragmentation must yield the same Request object, exact consumed length, and identical leftover bytes.

Layer 9 - Integrated request lifecycle

Join the event loop, parser, configuration, routing, work execution, output buffering, and cleanup into one explicit state machine.

L9-Q1. What states can one connection occupy?

Answer. A practical design may include reading headers, reading body, request complete, selecting route, reading a file, managing CGI, preparing response, writing response, waiting for the next request, closing, and failed. The exact names can differ, but every legal operation must follow from explicit state.

L9-Q2. What event causes each state transition?

Answer. Transitions follow concrete results: bytes complete a parser phase, route resolution chooses work, a regular file finishes, CGI pipes reach EOF and the child exits, an output offset reaches the buffer end, a deadline expires, or the peer disconnects. Avoid hidden transitions based on scattered booleans.

L9-Q3. When is a route selected?

Answer. After the request line and relevant headers are syntactically and semantically valid and the normalized path is available. Body limits may depend on the selected route, so selection should occur early enough to enforce them before full reception.

L9-Q4. How is the correct server configuration associated with each listening socket?

Answer. The listener registry stores the configuration identity alongside its descriptor. accept copies or references that identity into the new Connection, so later routing never has to infer the server from the descriptor number or Host alone.

L9-Q5. What data belongs to the connection, request, route, and response?

Answer. Connection owns transport state and buffers; Request owns parsed method, target, version, headers, and body metadata; Route owns configured policy; Response owns status, headers, body source, framing, and output progress. Keeping these roles clear prevents lifecycle data from leaking across requests.

L9-Q6. What is a response plan, and why should routing not immediately send bytes?

Answer. A response plan describes status, headers, body source, CGI or file work, and connection policy. The event loop later performs allowed I/O. This separates a pure decision from partial writes and makes accurate error substitution possible before output starts.

L9-Q7. How are parser errors converted into HTTP error responses?

Answer. The parser returns a structured error category without performing network I/O. The lifecycle maps it to an accurate status, selects a custom or default error page, decides whether the connection is safe to reuse, and queues the resulting response.

L9-Q8. When should a connection close after an error?

Answer. Close when message boundaries are ambiguous, the client requested close, the HTTP version or framing requires it, a timeout or transport failure occurred, or the server cannot safely parse another request. A clean route-level 404 or 405 can often preserve an otherwise valid persistent connection.

L9-Q9. How is a connection reset for another request without losing leftover bytes?

Answer. Destroy or reset request, route, work, response, and deadlines while preserving the client descriptor and unconsumed input suffix. Initialize a fresh parser over that suffix and update the read and write interests.

L9-Q10. How are input and output buffers owned?

Answer. The Connection should own transport buffers because they survive individual readiness events. Parser and response writer receive controlled access and offsets but must not retain invalid references after buffer modification.

L9-Q11. How are files, CGI pipes, and child processes attached to the originating request?

Answer. Store them in a request-scoped work context owned by the Connection, with each auxiliary descriptor registered back to that context. An event on a CGI pipe can then find the correct client and response state directly.

L9-Q12. What cleanup must occur from every possible state?

Answer. Remove poll registrations, close client and auxiliary descriptors, close directory or file resources, kill and reap a live CGI when necessary, delete incomplete temporary uploads, release buffers, and invalidate registry links. Cleanup should be centralized and idempotent.

L9-Q13. How do you prevent a response from being generated twice?

Answer. Make response creation a one-way state transition and record whether headers or body bytes have begun. Later errors either replace an unstarted plan or force closure; they must never append a second HTTP response into the middle of the first by accident.

Practical pass gate

Trace every state, descriptor, buffer, deadline, and cleanup action for six scenarios: successful static GET, missing file, forbidden method, upload, CGI POST, and a client disconnecting halfway through its request.

Layer 10A - Static website, routes, and methods

Implement the visible HTTP behaviour for files, directories, redirects, method rules, deletion, and error pages.

L10A-Q1. How is the longest matching route selected?

Answer. Compare the normalized request path against route prefixes and select the valid match with the greatest length, respecting path-segment boundaries. Use a deterministic tie rule and reject ambiguous duplicate configuration at startup.

L10A-Q2. How is the URL path transformed into a filesystem path?

Answer. Separate the query, validate and decode the path according to one policy, normalize its segments, remove the matched route prefix, append the remainder to the configured root, and verify that the final target remains inside that root.

L10A-Q3. In what order should percent-decoding and path normalization happen?

Answer. Parse the URI components first, validate percent escapes, decode the path exactly once, reject dangerous encoded separators or nulls according to policy, then normalize segments. Finally perform a containment check under the configured root; never normalize only the still-encoded string.

L10A-Q4. How will .., encoded separators, and embedded null bytes be prevented from escaping the configured root?

Answer. Reject embedded nulls and ambiguous separators, resolve dot segments after decoding, and compare the normalized final path against the normalized root boundary. Do not rely on string concatenation or substring prefix checks that confuse /root with /root2.

L10A-Q5. What happens when GET targets a regular file?

Answer. If routing, method, existence, and permissions allow it, determine metadata and media type, build a 200 response with correct body framing, and deliver the file. Open or read failure must become an appropriate error before response bytes are committed.

L10A-Q6. What happens when GET targets a directory with an index?

Answer. Look for the configured index file inside that directory. If it exists and is a permitted regular file, serve it; otherwise continue to the route's autoindex or error behaviour.

L10A-Q7. What happens when directory listing is enabled?

Answer. Read the directory entries and generate a safe HTML representation with escaped names and correctly encoded links. Exclude entries you deliberately do not expose and ensure every generated link remains under the request path.

L10A-Q8. What happens when directory listing is disabled?

Answer. If no usable index exists, return the configured refusal or not-found behaviour, commonly 403. The exact status should be consistent and compared with the reference behaviour chosen by the team.

L10A-Q9. What status and header are required for a redirect?

Answer. Return the configured 3xx status and a Location header containing the destination. Include correct framing for any optional explanatory body and apply the chosen connection policy.

L10A-Q10. What happens when a recognized method is forbidden for a route?

Answer. Return 405 Method Not Allowed and do not perform the operation. Include an Allow header listing methods the selected resource permits.

L10A-Q11. What must the Allow header contain?

Answer. It contains the set of methods currently supported for the selected resource or route, formatted as comma-separated method tokens. It should reflect actual behaviour, not merely every method Webserv recognizes globally.

L10A-Q12. What is your precise POST behaviour for static, upload, and CGI routes?

Answer. POST has no universal filesystem meaning. The server must define it by route: reject it when disallowed, store the body under an upload policy, or pass it to CGI. The configuration and test website should make that decision observable.

L10A-Q13. What can DELETE remove: files, empty directories, or both?

Answer. The subject requires DELETE but leaves detailed resource policy to the implementation. A conservative design supports configured regular-file deletion and explicitly documents whether directories are refused or only empty directories are allowed.

L10A-Q14. Which status code does each DELETE outcome produce?

Answer. A successful deletion can return 204 or 200 with a result body; a missing target is 404; forbidden access is 403; a policy conflict such as a non-empty directory can be 409; unexpected filesystem failure is 500. Use one documented matrix consistently.

L10A-Q15. How is Content-Type chosen?

Answer. Map known file extensions to media types and use a safe default such as application/octet-stream for unknown binary content. Generated HTML and error pages should explicitly use an HTML type and suitable charset.

L10A-Q16. How are custom error pages selected?

Answer. After choosing the final status, look up that code in the effective server error-page map. Serve the configured file if it can be read without causing recursive error generation, while preserving the original status code.

L10A-Q17. When is the built-in default error page used?

Answer. Use it when no custom page is configured or the custom file cannot be obtained safely. The fallback must always be available from generated text so error handling does not depend on another fragile disk operation.

L10A-Q18. How do you ensure every response has accurate status, headers, and framing?

Answer. Centralize response construction and validate invariants before queuing bytes: one status line, no conflicting body framing, correct Content-Length for buffered bodies, required Location or Allow fields, body restrictions for relevant statuses, and a clear keep-alive decision.

Practical pass gate

Create a route-and-status matrix for GET, POST, and DELETE against an existing file, missing file, directory, forbidden method, redirect, upload route, CGI route, permission failure, and invalid path. Test every matrix cell with raw assertions.

Layer 10B - Request bodies and file uploads

Receive large and binary bodies safely, enforce limits during streaming, and leave no corrupt partial files.

L10B-Q1. How is the maximum allowed body size selected for a request?

Answer. After normalized path routing, obtain the effective route or server body limit and attach it to request state. That same immutable value must govern declared length, chunk accumulation, buffering, upload writing, and CGI input.

L10B-Q2. When can an oversized Content-Length request be rejected immediately?

Answer. As soon as valid headers and the effective route are known, compare the checked numeric Content-Length with the configured maximum. If it exceeds the limit, return 413 without receiving the declared body, usually closing to avoid ambiguous remaining bytes.

L10B-Q3. How is the limit enforced when the request uses chunked encoding?

Answer. Track the cumulative decoded body size with overflow checks. Before accepting each chunk, verify that current size plus the parsed chunk size remains at or below the limit; otherwise reject with 413.

L10B-Q4. Will an upload be buffered in memory, streamed to disk, or use a hybrid strategy?

Answer. This is an architecture choice. Streaming or a bounded hybrid design scales better because body size does not become equivalent memory usage. Whatever is chosen must preserve incremental validation, binary bytes, cleanup, and CGI needs.

L10B-Q5. How will binary files containing null bytes be preserved exactly?

Answer. Use explicit byte counts for every append and file write, never strlen or text delimiters. Tests should compare hashes or complete byte arrays after upload, not only filenames and sizes.

L10B-Q6. What upload representation will your demo support: raw body, multipart form, or CGI processing?

Answer. The subject requires file upload but does not fully prescribe its HTTP representation. Document one demonstrable contract: a direct route storing the raw body, a server-side multipart parser, or a CGI that processes multipart form data. A browser form requires either multipart handling or CGI that understands it.

L10B-Q7. If multipart upload is supported directly, how are boundaries and filenames parsed?

Answer. Parse the boundary parameter from Content-Type, recognize boundaries incrementally across reads, parse each part's headers, and treat part data as binary. Extract filenames only from supported disposition syntax and sanitize them before path construction.

L10B-Q8. How are filenames sanitized against path traversal?

Answer. Discard directory components, reject nulls, separators, dot-only names, and invalid encodings, and generate a server-side name when possible. Always join against the configured upload directory and verify containment after normalization.

L10B-Q9. What happens when the destination file already exists?

Answer. Choose an explicit policy: reject with conflict, generate a unique name, or replace only when configuration authorizes it. Never overwrite silently because a client happened to choose an existing filename.

L10B-Q10. What happens if disk creation or writing fails?

Answer. Stop the upload, close the descriptor, remove any incomplete temporary file, and return an accurate server or permission error if response output has not started. The event loop must remain operational for other clients.

L10B-Q11. How is an incomplete file cleaned up when the client disconnects?

Answer. Write into a request-owned temporary path and record whether it has been committed. Connection cleanup closes the file and unlinks the temporary output unless the complete validated body was finalized successfully.

L10B-Q12. Which status code is returned after a successful upload?

Answer. 201 Created is the clearest choice when a new resource was stored, preferably with a Location value if it has a retrievable URI. A documented 200 result can be valid for other processing, but response semantics must match what occurred.

L10B-Q13. How is a zero-byte upload handled?

Answer. A zero-length file can be a valid upload when the route permits it and framing is valid. Create and commit an empty file or pass an empty body to the selected handler rather than confusing zero length with client disconnection.

L10B-Q14. Why must a chunked upload be decoded before its contents are stored?

Answer. Chunk sizes and CRLF delimiters are HTTP transfer metadata, not file bytes. Storage receives only the concatenated chunk data, and the cumulative decoded length is the actual uploaded content size.

Practical pass gate

Verify byte-for-byte results for an empty file, text file, binary file, exact maximum, one byte over, fragmented input, chunked input, disconnect halfway through, invalid destination, and duplicate filename. Confirm partial files and descriptors are always cleaned up.

Layer 10C - CGI

Run external programs only for configured CGI resources while keeping every pipe and child inside the non-blocking lifecycle.

L10C-Q1. What problem does CGI solve?

Answer. CGI defines how a web server starts an external program, describes the HTTP request through environment variables and standard input, and interprets the program's standard output as dynamic response metadata and content.

L10C-Q2. How does a route decide that a resource must be executed as CGI?

Answer. After safe path and route resolution, compare the script's extension with the route's configured CGI mappings. Only an existing permitted script with a known mapping enters CGI; unknown extensions follow static or error policy.

L10C-Q3. Which executable and script path are passed to `execve()`?

Answer. Use the interpreter or executable configured for the extension and provide an argv matching that program, commonly interpreter followed by script path. Do not rely on a shell; `execve` receives the exact executable, argument vector, and environment vector.

L10C-Q4. Which CGI environment variables must be supplied?

Answer. Provide the variables required by your CGI contract, typically `GATEWAY_INTERFACE`, `SERVER_PROTOCOL`, `REQUEST_METHOD`, `QUERY_STRING`, `CONTENT_LENGTH` when known, `CONTENT_TYPE` when present, `SCRIPT_NAME` or script path information, `SERVER_NAME`, `SERVER_PORT`, and relevant HTTP headers in CGI form.

L10C-Q5. How are method, query string, protocol, content type, and body length represented?

Answer. They become string environment entries such as `REQUEST_METHOD=POST`, `QUERY_STRING=a=1`, `SERVER_PROTOCOL=HTTP/1.1`, `CONTENT_TYPE=application/json`, and `CONTENT_LENGTH=12`. Values must be derived from the validated request, not raw untrusted lines.

L10C-Q6. How is the complete request body delivered to CGI standard input?

Answer. Webserv writes the decoded body bytes to the parent-to-child pipe connected to the child's descriptor 0. Partial non-blocking writes advance an offset until every byte has been delivered.

L10C-Q7. Why must the parent close the CGI input pipe to deliver EOF?

Answer. Many CGI programs read until EOF. Even after all body bytes are written, the child will keep waiting while any writer for that pipe remains open. Closing the parent's write end signals that no more input exists.

L10C-Q8. How is CGI standard output returned to Webserv?

Answer. Before `execve`, the child connects its standard output to the child-to-parent pipe. The parent registers the read end with the central multiplexer and incrementally collects or forwards CGI headers and body data.

L10C-Q9. Why must unused pipe ends be closed in both parent and child?

Answer. Unused open ends leak descriptors and can prevent EOF forever because the kernel still sees a writer or reader. Each process should keep only the ends it actually uses after `fork` and `dup2`.

L10C-Q10. Why must CGI run in the correct working directory?

Answer. CGI scripts may open templates or data using relative paths. The child should `chdir` to the configured script directory before `execve` so those references resolve as intended without changing the long-lived server parent's directory.

L10C-Q11. How are CGI headers separated from its response body?

Answer. Parse the CGI output header section until its empty-line delimiter, validate supported fields, then treat subsequent bytes as body content. Do not forward malformed CGI header bytes blindly as a normal HTTP response.

L10C-Q12. How is a CGI Status header handled?

Answer. If present and valid, it supplies the HTTP response status code and optional reason. If absent, normal CGI output generally implies 200 unless another defined CGI behaviour, such as a local redirect, applies.

L10C-Q13. What marks the end of CGI output when it provides no content length?

Answer. EOF on the CGI stdout pipe marks the end, exactly as the subject states. The parent must read until the pipe closes while also collecting the child's exit status.

L10C-Q14. How will Webserv frame CGI output for the HTTP client?

Answer. It can buffer the complete body and send `Content-Length`, stream using valid chunked transfer coding for an HTTP/1.1 client, or use connection closure where protocol rules allow. The chosen strategy must never send ambiguous boundaries on a persistent connection.

L10C-Q15. Why must CGI pipes be non-blocking and monitored by the same central poll?

Answer. A child may read or write slowly, and pipe buffers are finite. A blocking operation would freeze all clients. The subject explicitly classifies pipes as I/O that can wait, so they belong in the single readiness-driven mechanism.

L10C-Q16. Why can writing the entire request before reading CGI output deadlock?

Answer. The child may produce enough output to fill its stdout pipe before it has consumed all input. It then blocks writing while the parent blocks trying to finish input into another full pipe. Concurrent readiness handling breaks that cycle.

L10C-Q17. How can Webserv read CGI output and write CGI input concurrently?

Answer. Register both relevant pipe ends, request write readiness while input remains, and request read readiness until output EOF. Each event advances its own offset or buffer, and neither direction waits synchronously for the other to finish.

L10C-Q18. How is waitpid() used without blocking the server?

Answer. Call waitpid for known child PIDs with WNOHANG at safe event-loop points or after a signal indicates possible exits. A zero result means the child is still running; a positive PID provides the collected termination status.

L10C-Q19. What happens if the program does not exist or execve() fails?

Answer. The child must report failure through a controlled exit or dedicated error path and terminate. The parent detects it, closes pipes, reaps the child, and generates a server or bad-gateway style response if no HTTP bytes have been committed.

L10C-Q20. What happens if CGI crashes, hangs, or produces malformed output?

Answer. A crash is detected through pipe EOF and exit status; a hang through a deadline; malformed output through the CGI parser. Clean all resources and return an accurate 5xx response when still possible, without affecting other connections.

L10C-Q21. How are timed-out CGI processes killed and reaped?

Answer. On deadline expiry, send an allowed termination signal, close associated pipes, and call non-blocking waitpid until the child is collected, escalating according to the team's bounded policy if needed. Never abandon a live child or leave a zombie.

Practical pass gate

Test CGI programs that echo request metadata, produce a large response, omit Content-Length, sleep indefinitely, exit unsuccessfully, emit invalid headers, and simultaneously read slowly while writing heavily. Webserv must remain responsive and leave no pipe or child behind.

Layer 11 - Resilience, backpressure, and cleanup

Keep the server available when clients, files, children, memory, and network timing behave badly.

L11-Q1. How is the unsent portion of a response represented?

Answer. Keep immutable or stable queued bytes plus an offset to the first unsent byte, or an equivalent sequence of bounded chunks. Never erase or reallocate data in a way that invalidates the active send position.

L11-Q2. What happens when send() transmits only part of the output?

Answer. Advance the output offset by exactly the positive return value, leave the remainder queued, retain write interest, and return to the event loop. Do not retry from byte zero or busy-loop until completion.

L11-Q3. When should write readiness be enabled and disabled?

Answer. Enable it when a socket or pipe has pending bytes it is allowed to send. Disable it immediately when the queue becomes empty or the lifecycle is waiting for some other event, avoiding constant writable wake-ups.

L11-Q4. How does the server handle a slow-reading client?

Answer. Apply a bounded output queue, stop or slow upstream production, wait for write readiness, and enforce a response-progress deadline. Close stalled clients and clean their work before they consume unbounded memory.

L11-Q5. How does it handle a client that never finishes sending headers?

Answer. Track both header-size and header-completion time. Once either limit is exceeded, send an error if safely possible or close the connection, releasing its partial buffer.

L11-Q6. Which timeouts exist for headers, bodies, responses, and CGI?

Answer. Use separate deadlines or progress timers because each phase has different expectations: header completion, body progress, output progress, and CGI execution or I/O. The configuration may use common values, but state must identify which deadline expired.

L11-Q7. What limits prevent one connection from consuming unlimited memory?

Answer. Bound request line, headers, body, parser leftovers, response queue, CGI output, and pending requests. Prefer streaming for large bodies and files, and close work that cannot respect the configured limits.

L11-Q8. How is SIGPIPE prevented from terminating the server?

Answer. Ignore SIGPIPE with the allowed signal interface or use an available safe send mechanism consistent with the subject, then treat the failed write as a connection failure. A peer closing early must never terminate the whole process.

L11-Q9. How are sockets, files, pipes, and child processes cleaned up after every failure?

Answer. Route all exits through owner-based cleanup that unregisters events, closes each owned descriptor once, removes temporary files, terminates and reaps children when required, and clears cross-references. Test cleanup from every lifecycle state.

L11-Q10. How will descriptor leaks be detected?

Answer. Run repeated and stress scenarios while observing the process descriptor count through platform tools such as `/proc`, `lsof`, or a debugger. The count should return to its stable baseline after clients and CGI jobs finish.

L11-Q11. How are zombie processes detected?

Answer. Inspect process state during repeated CGI execution and verify every known PID is eventually collected by `waitpid`. A growing list of defunct children means exit handling or cleanup paths are incomplete.

L11-Q12. How does the loop remain fair with many active clients?

Answer. Limit work per descriptor per iteration, rotate through all reported events, avoid unbounded accept or send loops, and keep parsing and generation bounded. A continuously ready client must not prevent later entries from being handled.

L11-Q13. What happens when a client disconnects while its CGI is still running?

Answer. Detach no unmanaged work: close the client, stop feeding input, terminate the child according to policy, close its pipes, reap it, discard unneeded output, and remove all event registrations.

L11-Q14. Can failure serving one request terminate or corrupt another connection?

Answer. It must not. Per-connection ownership, structured errors, bounds checks, and controlled exception boundaries isolate failure. Only unrecoverable startup or global infrastructure failure should stop the server deliberately.

L11-Q15. How does the server guarantee that a request never hangs indefinitely?

Answer. Every waiting state has a deadline and the central poll has a finite timeout derived from pending deadlines. Expiry triggers a defined response or closure and complete resource cleanup.

L11-Q16. How are allocation failures and unexpected exceptions contained?

Answer. Catch them at request and event-loop boundaries, mark the affected context failed, release resources with `RAII` or centralized cleanup, and avoid recursive allocation-heavy error handling. Log minimally and continue when global state remains consistent.

Practical pass gate

Run normal clients, slow senders, slow readers, malformed requests, large responses, uploads, CGI jobs, and abrupt disconnects at the same time. The server must keep accepting new clients, enforce deadlines, and return descriptor, process, and memory use to a stable baseline.

Layer 12 - Integration and stress testing

Prove each feature independently and then prove the complete server remains correct under fragmentation, concurrency, and failure.

L12-Q1. Which behaviours require unit tests?

Answer. Test pure or isolated logic such as configuration grammar, route matching, path normalization, header parsing, fixed and chunked framing, status selection, MIME lookup, response serialization, and state transitions with deterministic inputs.

L12-Q2. Which behaviours require full integration tests?

Answer. Socket readiness, partial I/O, multiple listeners, browser behaviour, filesystem effects, upload cleanup, CGI pipes and process lifecycle, timeouts, disconnects, persistent connections, and stress all require a running server.

L12-Q3. How will a test deliberately fragment a request across writes?

Answer. Use a raw socket client that sends selected byte slices separately, optionally waiting between them. Test every meaningful split and one-byte delivery rather than relying on the operating system to fragment traffic by chance.

L12-Q4. How will malformed messages be generated?

Answer. Construct raw byte sequences with invalid request lines, line endings, header names, lengths, duplicate framing, chunk syntax, oversized sections, embedded controls, incomplete bodies, and early close. High-level HTTP libraries often refuse to send these cases.

L12-Q5. How will every configuration directive be demonstrated?

Answer. Maintain a coverage matrix linking each directive to a configuration block, test URL, expected status, headers, body or filesystem side effect, and an invalid-configuration case. Evaluation files should make the same matrix easy to show manually.

L12-Q6. How will all configured ports be tested?

Answer. Connect to every interface-and-port pair, assert its distinct route content and limits, and run clients against ports concurrently. Also test startup conflicts and unavailable bind addresses.

L12-Q7. How will status codes, headers, and bodies be asserted separately?

Answer. Parse the response in the test client and compare the numeric status, required and forbidden headers, exact framing, and exact body bytes independently. A page looking correct in a browser does not prove the protocol metadata is correct.

L12-Q8. How will browser compatibility be verified?

Answer. Load the complete static test website, follow redirects, fetch linked assets, submit uploads or forms, and inspect the browser network panel for failed requests, incorrect MIME types, malformed responses, or unexpected connection closure.

L12-Q9. How will telnet or a raw socket reveal behaviours hidden by curl?

Answer. It exposes exact bytes and permits incomplete, delayed, or malformed messages that curl normalizes or refuses. This is essential for testing incremental parsing, timeouts, connection reuse, and error recovery.

L12-Q10. What should be compared with NGINX?

Answer. Compare route mapping, index and autoindex behaviour, redirects, common status codes, method rejection, error pages, headers, and selected malformed cases. Record the exact NGINX and HTTP configuration used.

L12-Q11. Why should differences caused by HTTP versions not be copied blindly?

Answer. NGINX may use HTTP/1.1 or newer policies and implement far more RFC behaviour than the assignment. Understand whether a difference comes from version, configuration, optional features, or a Webserv bug before changing code.

L12-Q12. How will file-descriptor and memory leaks be measured?

Answer. Record stable baselines, run many repeated clients and CGI jobs, then observe descriptors, child processes, heap diagnostics, and resident memory after work ends. Investigate growth rather than accepting that the process is long-lived.

L12-Q13. What does the server remains available mean in a stress test?

Answer. While many or hostile clients are active, new well-formed clients must still connect and receive correct bounded-latency responses. After the load stops, the same server process must continue working without restart.

L12-Q14. Why is the supplied tester insufficient by itself?

Answer. No small external tester can know every configuration, platform detail, fragmentation, browser interaction, CGI failure, resource leak, or team design choice. The subject explicitly asks for varied clients and your own tests.

L12-Q15. Can every mandatory feature be demonstrated from a clean machine?

Answer. It should compile and run from a clean clone using repository-contained configurations, static files, error pages, CGI examples, and documented commands. Hidden local paths or installed-only assets make the submission incomplete.

Practical pass gate

Create an automated suite for valid and malformed requests, every method and route directive, fragmented and chunked messages, multiple ports, static files, uploads, CGI, timeouts, disconnects, persistence, concurrency, and stress. Then reproduce the mandatory demonstrations manually from a clean clone.

Layer 13 - Submission and defense

Deliver a reproducible C++98 project and prove that every teammate understands and can modify it.

L13-Q1. Does the project compile with `c++ -Wall -Wextra -Werror -std=c++98`?

Answer. It must compile cleanly with those constraints. Test the exact command and a clean rebuild; compiler defaults or later-language extensions must not be required.

L13-Q2. Do all required Makefile rules work without unnecessary relinking?

Answer. Run NAME or all, clean, fclean, and re from clean and built states. Running make twice without changes should perform no compile or link command.

L13-Q3. Does `./webserv [configuration file]` behave correctly?

Answer. With one valid argument it must parse that file and start the configured listeners. Invalid counts, malformed files, and startup failures must produce controlled diagnostics and no partially running server.

L13-Q4. Does a valid default configuration work when the argument is omitted?

Answer. If the team supports the subject's default-path option, the documented file must exist and demonstrate required features. Otherwise follow the chosen interpretation consistently and make evaluation invocation unambiguous.

L13-Q5. Are only authorized functions and libraries used?

Answer. Audit includes, link flags, wrappers, scripts, and transitive dependencies. External and Boost libraries are forbidden, and system interactions must stay within the subject's authorized function list and associated permitted macros.

L13-Q6. Can you prove that `fork()` is used only for CGI?

Answer. Search every call site and explain its path from a configured CGI request. Client concurrency, file service, uploads, and general error handling must remain in the single event-driven server process.

L13-Q7. Are demonstration configurations and default files included?

Answer. The repository should contain configurations, static sites, index files, custom error pages, upload directories or setup instructions, and at least one CGI program that collectively demonstrate every mandatory directive and feature.

L13-Q8. Is the README written in English?

Answer. Yes; the subject explicitly requires English so peers, staff, and recruiters can understand the project and reproduce it.

L13-Q9. Is its first line exactly the required italicized sentence with the team logins?

Answer. The first line must be italicized and read: This project has been created as part of the 42 curriculum by the listed login or logins. Preserve the required wording and replace placeholders correctly.

L13-Q10. Does it contain Description, Instructions, and Resources?

Answer. Description explains the goal and overview; Instructions give compilation, setup, configuration, execution, and test usage; Resources list relevant references. Additional architecture and feature sections make the project easier to evaluate.

L13-Q11. Does it describe precisely how AI was used?

Answer. State which tasks used AI, such as explanations, test ideas, or review, and which project areas were affected. Do not claim generic use; the team remains responsible for understanding, checking, and defending every result.

L13-Q12. Can every teammate explain the event loop, parser, routing, uploads, and CGI?

Answer. Every teammate should be able to explain data flow, invariants, key structures, error paths, and subject compliance even for code primarily written by someone else. Evaluation tests collective ownership, not task assignment history.

L13-Q13. Can every teammate trace one request through the complete program?

Answer. They should start at listener readiness and accept, follow reads, parsing, configuration and route selection, file or CGI work, response construction, partial sends, persistence or closure, and resource cleanup.

L13-Q14. Can you make a small behavioural modification during the evaluation?

Answer. The subject permits a brief requested change. Clear separation, focused tests, and shared understanding should let the team locate the responsible component, implement the change, rebuild, and verify it within minutes.

L13-Q15. Has another person performed a clean peer review?

Answer. A reviewer should compile from scratch, use unfamiliar configurations and raw clients, inspect subject compliance and functions, stress failure paths, and question architectural decisions. Familiar team tests alone leave blind spots.

Practical pass gate

From a clean clone, compile, launch, demonstrate every mandatory feature, run malformed and stress tests, explain any selected line, and complete a small behavioural change without restructuring the project.

Bonus - Only after mandatory behaviour is flawless

Treat cookies, sessions, and additional CGI types as isolated extensions, never as compensation for missing mandatory reliability.

B-Q1. What information is stored in a cookie?

Answer. A cookie is a small name-value item plus attributes sent by the server and later returned by the client under matching scope rules. It can carry a session identifier, but sensitive server-side state should not normally be trusted directly from an unsigned client value.

B-Q2. What distinguishes a cookie from server-side session state?

Answer. The cookie lives at the client and is attached to requests. Session state lives at the server and is looked up using an opaque identifier from the cookie. Deleting either side can invalidate the association.

B-Q3. How are session identifiers generated, validated, and expired?

Answer. Use sufficiently unpredictable unique values, accept only the exact expected syntax, bind them to a session record, and enforce idle and absolute expiry. Unknown or expired identifiers create a new session rather than indexing arbitrary memory or files.

B-Q4. How are session fixation and predictable identifiers avoided?

Answer. Do not let clients choose valid identifiers, regenerate an identifier when privilege changes, use strong randomness, and invalidate old records. Sequential counters and timestamps alone are predictable.

B-Q5. How will multiple CGI types select different executables?

Answer. Extend the validated extension-to-interpreter mapping so each configured type builds its own exact argv and environment while reusing the same non-blocking pipe, timeout, output-parser, and cleanup machinery.

B-Q6. Can the bonus be completely disabled without affecting mandatory behaviour?

Answer. It should be an additive configuration path. With bonus directives absent, routing, parsing, event handling, CGI baseline, tests, and resource ownership must behave exactly as the stable mandatory implementation.

Practical pass gate

Disable every bonus and rerun the complete mandatory suite. Then demonstrate cookie exchange, session creation and expiry, invalid-session handling, and two CGI interpreter mappings without leaks or changes to core behaviour.

Reference spine

The primary source is the Webserv subject, version 24.1. For deeper study, use the system manual pages for every allowed function, RFC 9110 for HTTP semantics, RFC 9112 for HTTP/1.1 message syntax and framing, RFC 3986 for URI syntax, RFC 3875 for CGI, and NGINX as a behavioural comparison rather than an unquestioned oracle.

The decisive test is understanding: you should be able to trace a request from an accepted socket through parsing, routing, file or CGI processing, buffered output, cleanup, and the next event-loop iteration.